

Week 12：MQTT多裝置、資料庫與可追蹤證據

日期：2026-11-25

本章讓多片ESP32透過同一個訊息轉送程式交換資料。送出訊息叫發布（publish），登記想接收的主題叫訂閱（subscribe），這種通訊方式使用MQTT協定。每片ESP32用自己的device_id區分主題，傳送光線讀值、事件及上線 / 離線狀態，也接收自己的命令。資料仍保存到Week 11的資料庫，手機可查看並操作指定裝置。

按裝置代號傳送訊息並保存資料

先在筆電的兩個視窗間傳一則訊息，再讓 ESP32 使用相同方式傳資料。手機頁面仍沿用 Week 11，最後確認關掉網頁後，資料還留在資料庫。

先練筆電：發布視窗 C → 訊息轉送程式 A → 訂閱視窗 B
再接裝置：ESP32 → 同一個 A → 轉接程式 E → 後端 D → 資料庫 / 手機

A 是 MQTT broker，負責按主題轉送訊息；E 是本課的 bridge，負責把 MQTT 訊息送進既有後端。兩者不是同一個程式。單一裝置完成傳送、回覆與保存後，多裝置測試才有可比較的結果。

一、本週內容

教學目標

完成本單元後，學生應能：

1. 解釋並示範訊息佇列遙測傳輸協定（MQTT）中的訊息代理伺服器（broker）、發布者（publisher）、訂閱者（subscriber）、主題（topic）、訊息內容（payload）、工作階段（session）、保留訊息（retained message）與遺囑訊息（Last Will）的角色。
2. 設計可隔離多個裝置的主題樹（topic tree），並區分遙測資料（telemetry）、事件（event）、在線狀態（presence）、命令（command）與回覆確認（acknowledgement）。
3. 從ESP32發布已驗證的感測資料與在線狀態，且只訂閱指定給該裝置的命令。
4. 使用穩定的命令識別碼（command_id）追蹤命令及其回覆，依文件中的記憶體快取（in-memory cache）機制，避免同次開機、最近8筆快取內的相同命令重複執行。
5. 用可觀察證據示範上線（online）、非預期離線（unexpected offline）、重新連線（reconnect）、錯誤裝置（wrong-device）、格式錯誤訊息（malformed payload）及代理伺服器無法使用（broker unavailable）的行為。
6. 將MQTT訊息接入既有後端（backend）、資料庫（database）、網頁雙向通訊（WebSocket）及行動介面（mobile interface），而不洩漏代理伺服器的認證資料（credential）。
7. 依裝置、事件與時間查詢SQLite歷史資料，解釋欄位、單位、空值及命令結果。

8. 區分感測無效、API拒絕與資料未送達，以相同識別碼及結構化紀錄重建原因。

教學內容

本單元介紹訊息佇列遙測傳輸協定 (MQTT)，以訊息代理伺服器 (broker) 協助發布 / 訂閱 (publish/subscribe)，支援多裝置物聯網通訊 (multi-device IoT communication)。學生會建立一致的主題階層 (topic hierarchy)，區分訊息路由 (message routing) 與JSON訊息內容 (JSON payload)，並使用在線狀態 (presence) 及回覆確認 (acknowledgement) 讓裝置狀態與命令結果可被觀察。實驗是在既有HTTP後端 (HTTP backend) 上擴充，不是取代它：MQTT橋接程式 (MQTT bridge) 驗證裝置訊息，透過相同的應用程式介面 (API) 儲存資料，再將手機命令轉送到各裝置專用的主題 (topic)。學生會測試保留的在線狀態、遺囑訊息 (Last Will)、重新連線 (reconnection)、重複命令防護 (duplicate command protection) 與本機安全行為。

後半段沿用同一批訊息與命令，不重新建立另一套系統。學生將它們對照到SQLite綱要 (schema)、歷史查詢、統計與結構化紀錄 (structured log)，以一條從裝置到資料列的證據鏈說明傳遞與儲存各自是否成功。

本週器材外觀

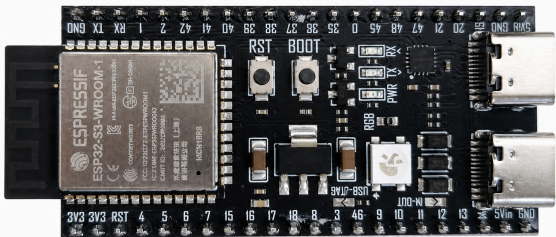
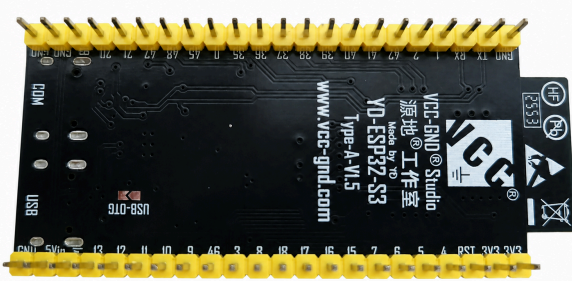
沿用兩顆 START / STOP 按鈕，加入光敏資料，透過 MQTT 保存紀錄並核對 RGB 實體結果。另備筆電、USB 資料線、區域網路與手機。

照片下方標示拍攝角度與來源。先辨認零件，再依本週器材表與接線步驟操作；照片本身不是接線指令，也不表示已完成電氣驗證。

[ESP32-S3 開發板](#) · [400 孔麵包板](#) · [杜邦線](#) · [四腳輕觸按鈕](#) · [KY-018 光敏電阻模組](#) · [HW-479 三色發光二極體模組](#)

ESP32-S3 開發板 (Development Board)

照片中的板卡為 YD-ESP32-S3 Type-A V1.5，搭載 N16R8 模組。正反面白底圖是既有後製展示圖；小字與腳位須核對本人實物及本週接線資料。

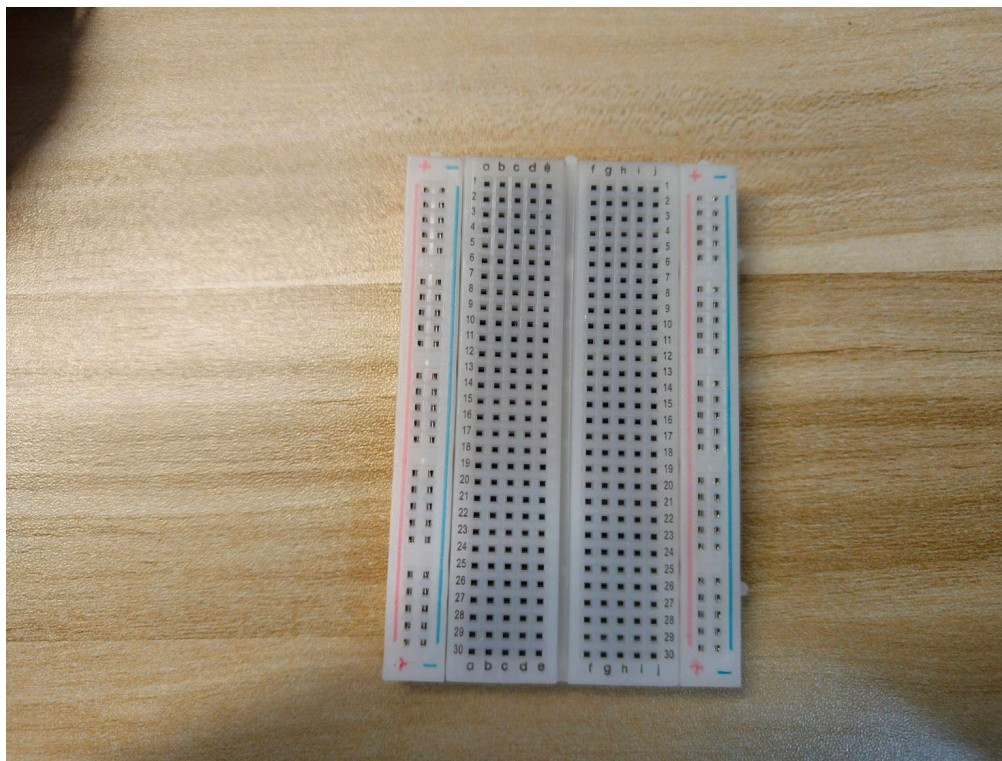
實物後製展示圖：正面：模組、按鈕與 USB 接頭	實物後製展示圖：背面：板身與排針
	

其他留存角度：[麵包板對孔紀錄](#)；不是建議的實驗安裝方式，右側接線空間不足。

400 孔麵包板 (Breadboard)

辨認中央溝槽、a~j 字母與列號。外觀照片不表示所有孔都相通，連通關係依本週圖解與斷電量測確認。

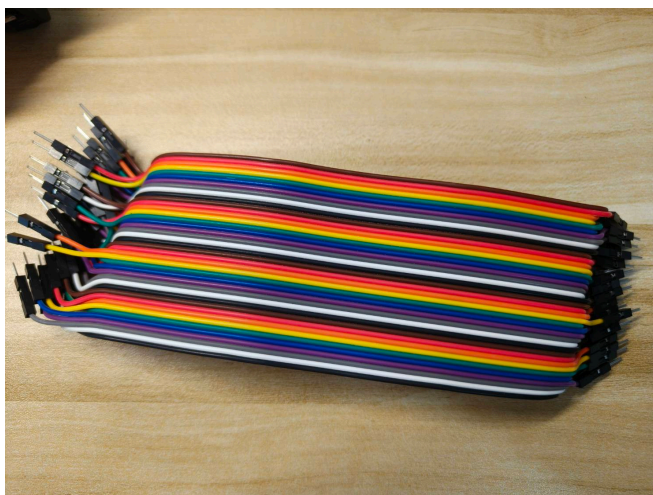
實物照片：俯視：中央溝槽、五孔組與側邊電源軌



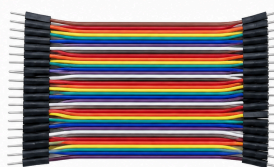
杜邦線 (Jumper Wire)

露出金屬針的是公頭 (Male)，有插孔的是母頭 (Female)；線色不會自行決定電壓或功能。所需接頭種類依當週器材表，不是每週都用完三種。商品參考卡上的數量與金額是歷史資料，不是學生應買數量或目前售價。

實物照片：成排導線與接頭全貌

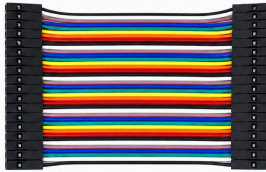


蝦皮商品參考：公對公：兩端皆為金屬針



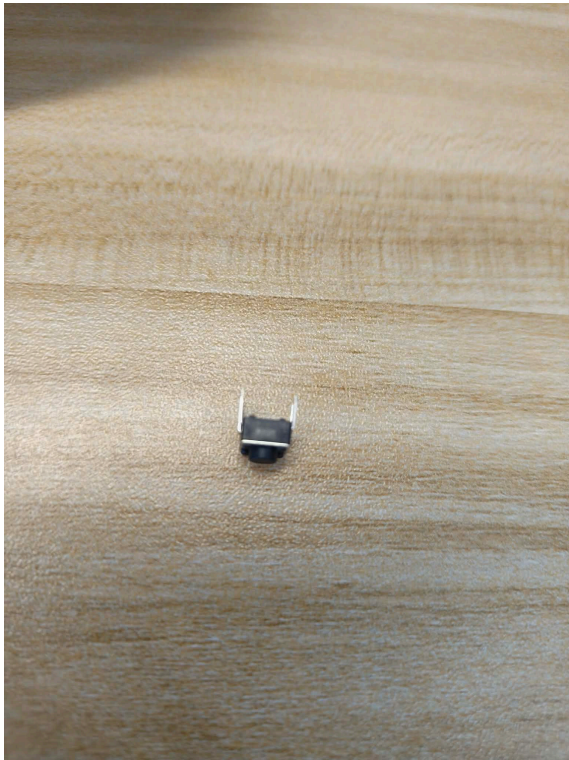
【環島科技】40P彩色杜邦線
線長10/20/30CM 一排40根
母頭對母頭 公頭對公頭
公頭對母頭彩色排線

規格：20cm，公對公
購買數量：6排
訂單金額：NT\$150

蝦皮商品參考：公對母：金屬針與插孔各一端	蝦皮商品參考：母對母：兩端皆為插孔
 【環島科技】40P彩色杜邦線 線長10/20/30CM 一排40根 母頭對母頭 公頭對公頭 公頭對母頭彩色排線 規格：20cm，公對母 購買數量：6排 訂單金額：NT\$150	 【環島科技】40P彩色杜邦線 線長10/20/30CM 一排40根 母頭對母頭 公頭對公頭 公頭對母頭彩色排線 規格：20cm，母對母 購買數量：6排 訂單金額：NT\$150

四腳輕觸按鈕 (Tactile Pushbutton)

上方黑色部分是按壓位置，四支金屬腳用來連接電路。照片不能單獨證明哪一對腳常通；先斷電，依 Week 2 的方法辨認。

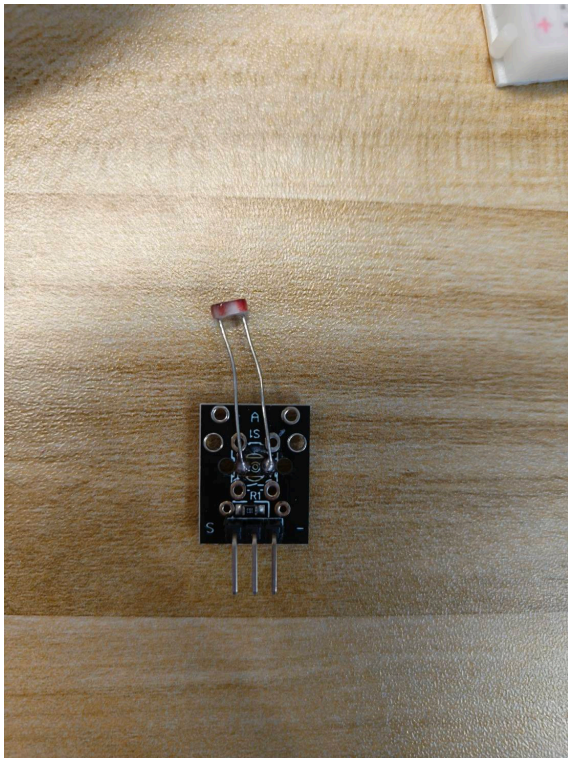
實物照片：俯視：按鍵與金屬上蓋	實物照片：側面：四支接腳
	

其他留存角度：[歷史接線紀錄](#)：同一組常通接點的量測，不是按下才導通的接法答案。

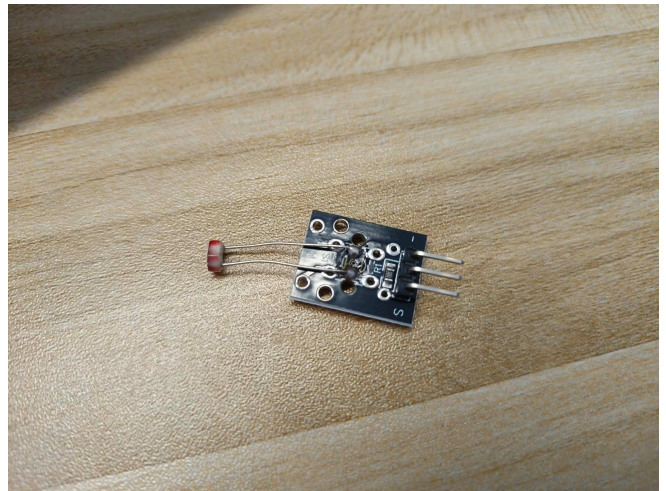
KY-018 光敏電阻模組 (Photoresistor Module)

圓形感光元件、板上固定電阻與三支排針構成模組。S 是訊號標示；元件區的 A、S1、R1 不能直接當成中間排針名稱。接線沿用已確認的 Week 3 紀錄。

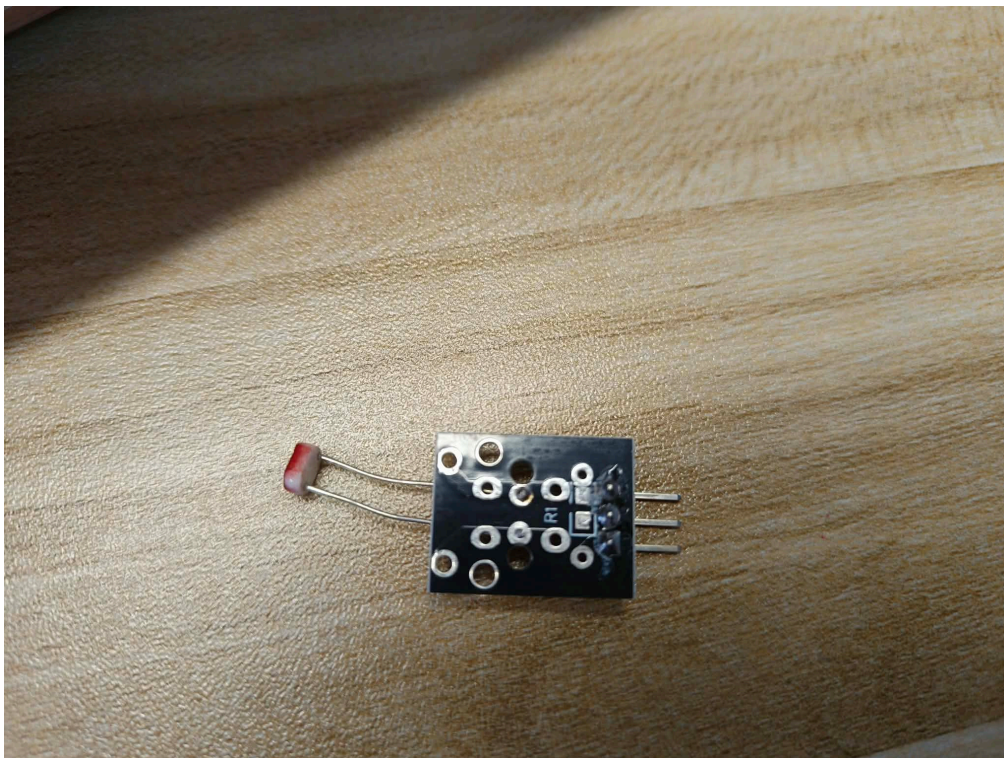
實物照片：正面近照：感光元件、S 與 - 絲印



實物照片：另一元件面角度

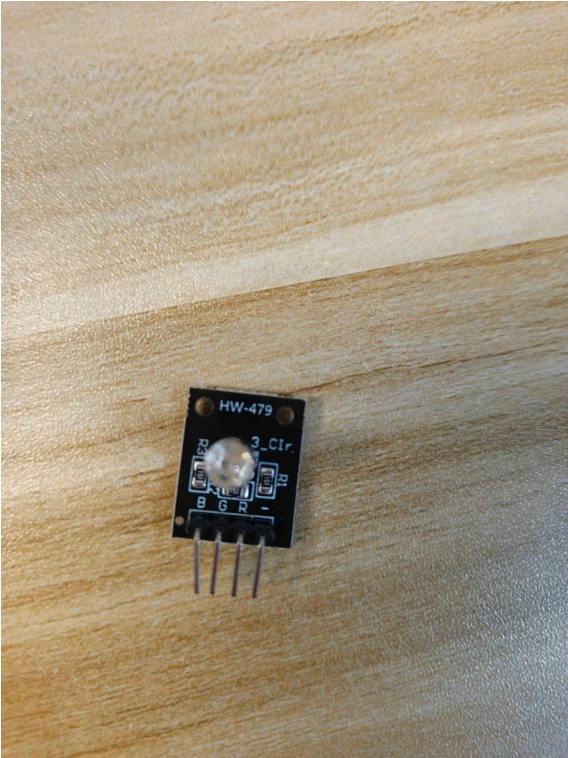
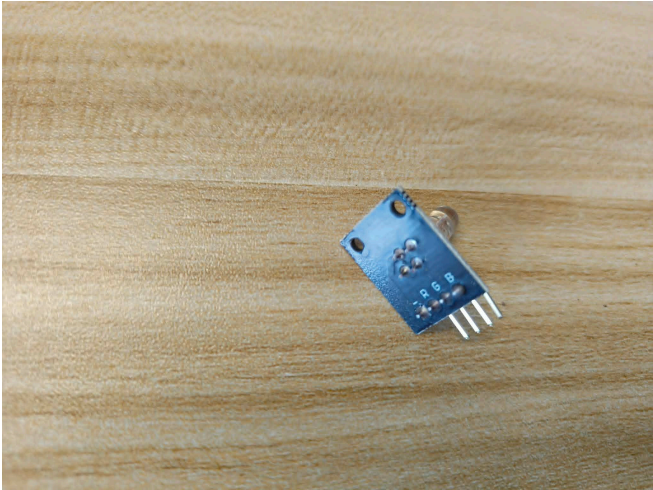


實物照片：焊接面



HW-479 三色發光二極體模組 (RGB LED Module)

訂單稱 KY-016；實物 PCB 標示 HW-479，前方可見 B、G、R、－ 與板上電阻。共同端、阻值及控制電流仍須核對，不能用八顆燈條取代這個模組。

實物照片：正面：單顆 LED 與 B / G / R / － 標示	實物照片：焊接面
	

其他留存角度：[較早的元件面照片](#)；[失焦補充照](#)；[不供腳位或焊點判讀](#)。

二、從Week 11架構加入Broker

MQTT broker (訊息代理伺服器) 是轉送訊息的程式：接收發布者 (publisher) 送來的訊息，再按主題 (topic) 轉給已訂閱該主題的訂閱者 (subscriber)。例如ESP32發布光線資料，監看視窗訂閱後就能收到；發布者不必知道接收者的IP。



topic (主題) 是broker用來路由訊息的階層名稱；**payload (承載資料)** 是topic內實際 傳送的內容，本課使用JSON。topic決定「送到哪一類接收者」，payload說明「這筆訊息的 欄位和值」。本課固定topic tree：

```
course/<device_id>/telemetry
course/<device_id>/events
course/<device_id>/presence
course/<device_id>/commands
course/<device_id>/acks
```

telemetry傳感測讀值，events傳事件，presence表示上線 / 離線，commands傳命令，acks傳回覆。斜線/分隔名稱層次；例如course/A/telemetry是A的感測資料主題。

訂閱篩選中的萬用字元+配對一層，#配對其所在位置的零層或多層名稱；例如course/+/telemetry可看各裝置的感測資料。<device_id>不得含/、空白、+或#，避免改變主題意義。

三、器材、軟體與安全邊界

本週使用ESP32-S3、USB資料線、麵包板、杜邦線、KY-018、START與STOP按鈕、KY-016 RGB、筆電、手機及可信任LAN。沿用Week 3、5、7已驗證的GPIO、光敏有效範圍與RGB ON level。本週不接SG90、蜂鳴器與4AA電池盒；broker斷線時實體STOP仍由本機處理，不以 broker成功作為前置條件。

- 1. Broker只在本週可信任LAN中使用，不設定router port forwarding。
- 2. 不使用公開測試broker傳送課程裝置資料或命令。
- 3. Broker使用臨時帳密；Wi-Fi與broker密碼放在secrets.h，不得提交Git。
- 4. 本課LAN範例沒有TLS，不可當成Internet部署。跨Internet還需TLS、裝置身分、憑證更新、細部授權、rate limiting與監控。
- 5. retained message是broker替主題保留、供後來訂閱者接收的最後一筆訊息。控制命令不使用它，避免新連線裝置執行過期命令。

四、安裝並驗證本機Broker

課前依[Eclipse Mosquitto官方下載頁](#)安裝Windows x64 版本。安裝位置與版本記於[Week 12支援資料](#)。

先準備視窗，不急著接 ESP32

本段 ESP32 先拔 USB。開五個 PowerShell 分頁或視窗，依序當作 A、B、C、D、E；以下每段都會標示使用哪一個。命令執行後若停留顯示 log，通常是正在等訊息，不是當掉。不要把下一個程式的命令貼進仍在執行的視窗。

視窗	用途	下一步是否保持執行
A	broker，轉送 MQTT 訊息	是，除非要換設定
B	訂閱，顯示收到的訊息	是，除非要換主題或帳密

視窗	用途	下一步是否保持執行
C	發布一則測試訊息	送完即回到提示符號
D	Week 11 的後端	是
E	MQTT 與後端的轉接程式	是

在 A 執行 `Test-Path 'C:\Program Files\mosquitto\mosquitto.exe'`，應為 `True`。若為 `False`，先在檔案總管找到實際安裝位置，再替換本節命令的路徑。若啟動顯示連接埠已占用，先查看是不是上一個實驗的 broker；不要重複啟動或任意結束不明程序。若是安裝時已啟動的 Windows 服務，須由有權限管理該服務的人處理後再繼續。

4.1 只限本機的host test

PowerShell視窗A：

```
& "C:\Program Files\mosquitto\mosquitto.exe" -v
```

保持A開啟。PowerShell視窗B訂閱：

```
& "C:\Program Files\mosquitto\mosquitto_sub.exe" `
-h 127.0.0.1 -t "course/host-test/events" -v
```

PowerShell視窗C發布：

```
$payload = '{"device_id":"host-test","event_type":"broker_test","value":1}'
$payload | & "C:\Program Files\mosquitto\mosquitto_pub.exe" `
-h 127.0.0.1 -t "course/host-test/events" `
-s
```

預期 B 顯示以下一行；A 顯示發布者及訂閱者連線：

```
course/host-test/events {"device_id":"host-test","event_type":"broker_test","value":1}
```

這只證明筆電內的 broker 訊息轉送，不證明 LAN、ESP32 或帳號設定。若 B 沒字，先核對 B 與 C 的 topic 是否完全相同，以及 A 是否仍執行。

`-s` 從管線讀取整段文字；如此 JSON 的雙引號不會因不同 PowerShell 原生命令傳參方式消失。

4.2 建立有帳密的LAN設定

先在 B 按 **Ctrl+C** 停止訂閱，再在 A 按 **Ctrl+C** 停止舊 broker。在 A 逐行執行，建立位於個人本機、不在課程 Git 資料夾內的練習目錄：

```
$mqttWork = Join-Path $env:LOCALAPPDATA 'IoTCourse\mqtt-practice'
New-Item -ItemType Directory -Force -Path $mqttWork
Set-Location -LiteralPath $mqttWork
Get-Location
Test-Path .\course-passwords
```

最後一行若為 **True**，表示已有密碼檔，不要再次用 **-c** 蓋掉。第一次為 **False** 才執行下列命令；密碼需輸入兩次，輸入時不顯示字元是正常現象：

-c 會覆寫既有檔案，依 [Mosquitto 官方命令說明](#)。

```
& "C:\Program Files\mosquitto\mosquitto_passwd.exe" -c .\course-passwords iotstudent
```

建立 **course-mosquitto.conf**，把 **password_file** 換成剛才檔案的完整實際路徑，路徑 使用正斜線：

在 A 輸入 **(Resolve-Path .\course-passwords).Path**，取得路徑；不要開啟或截圖密碼檔內容。再輸入 **notepad .\course-mosquitto.conf**，同意建立新檔，貼入下列五行，修改 **password_file** 後按 **Ctrl+S** 並關閉記事本。不要存成 **.conf.txt**。

```
listener 1883
allow_anonymous false
password_file C:/replace/with/actual/path/course-passwords
persistence false
log_type all
```

回到 A，先用 **Test-Path .\course-mosquitto.conf** 確認為 **True**，再啟動 LAN 設定：

```
& "C:\Program Files\mosquitto\mosquitto.exe" -c .\course-mosquitto.conf -v
```

Windows Firewall 只允許本次可信任的 **private network**，不關閉防火牆。在 B 用 **ipconfig** 取得筆電 LAN IPv4，再執行下列訂閱命令；把 **<筆電LAN-IP>** 改成實際數字、不留尖括號，密碼也替換成剛設定的值：

```
& "C:\Program Files\mosquitto\mosquitto_sub.exe" `
-h <筆電LAN-IP> -p 1883 -u iotstudent -P '<temporary-password>' `
-t "course/+presence" -v
```

不要截圖或提交含 **-P** 的命令。支援資料只記驗證結果，不抄密碼。

此時 ESP32 還沒上線，B 沒有新訊息是正常的，不能只靠空白畫面判斷登入成功。在 C 執行下列命令，替換成與 B 相同的筆電 IP 與密碼：

```
& "C:\Program Files\mosquitto\mosquitto_pub.exe" `
-h <筆電LAN-IP> -p 1883 -u iotstudent -P '<temporary-password>' `
-t "course/host-test/presence" -m 'online'
```

B 應顯示 `course/host-test/presence online`。這是文字連線測試，不是裝置的正式 presence JSON，也不要加 `-r` 留存它。

五、啟動Backend與MQTT Bridge

Bridge訂閱裝置telemetry、event、presence與ack，驗證JSON及topic中的device ID一致後，送入Week 11 Backend；它也查詢Backend的requested命令，發布到該裝置的commands topic，而且不retain。

在檔案總管找到 `IOT_Introduction/examples/course_backend`，從該資料夾開啟 D 與 E。兩個視窗都先執行 `Test-Path .\app.py` 與 `Test-Path .\venv\Scripts\python.exe`，都應為 True；缺 .venv 時回到 Week 11 第五節安裝，不在錯誤資料夾重建環境。

PowerShell 視窗 D 啟動後端。直接呼叫 .venv 裡的 Python，不需要啟用腳本：

```
$env:IOT_OPERATOR_KEY="replace-with-your-temporary-classroom-key"
.\venv\Scripts\python.exe -m uvicorn app:app --host 0.0.0.0 --port 8000
```

PowerShell視窗E同樣進入該資料夾：

```
$env:MQTT_HOST="<筆電LAN-IP>"
$env:MQTT_PORT="1883"
$env:MQTT_USERNAME="iotstudent"
$env:MQTT_PASSWORD="<temporary-password>"
$env:IOT_API_BASE_URL="http://127.0.0.1:8000"
.\venv\Scripts\python.exe mqtt_bridge.py
```

正常看到 `mqtt_connected`。不合法topic、無效JSON或device ID不一致會產生 `mqtt_message_rejected`，不寫入database。

若 E 顯示連不上 broker，先查 A 與 MQTT 帳密；若 MQTT 已連上但資料轉送失敗，先查 D。A、D、E 三個視窗必須同時保持執行。關掉 E 不等於關掉 A。

六、QoS、Retained Message與Last Will

QoS (Quality of Service) 指MQTT訊息的傳遞等級：0至多一次，不做MQTT層確認重送；1至少一次，可能重複收到。本週PubSubClient發布使用QoS 0，訂閱可要求QoS 1；bridge發布命令使用QoS 1。這些等級處理訊息傳遞，不保證實體動作完成，所以仍須以 `command_id`避免重複動作。PubSubClient發布限制與buffer設定見 [官方repository](#)。

retained message (保留訊息) 是broker替topic保存的最後一筆資料，新subscriber一訂閱就會收到。它適合presence，不適合一次性控制命令。

MQTT工作階段 (session) 保存客戶端的訂閱等通訊狀態；是否跨重連保留取決於連線設定，不能當作資料庫歷史。

Last Will (遺囑訊息) 由client連線時預先交給broker。若ESP32未正常告別就斷線，broker代為發布offline。online與意外offline都retain，監看者才可立即看到最後狀態。Will payload是在MQTT連線建立時準備，不是在斷線瞬間重新讀取裝置狀態。因此value=offline可作為broker觀察到非正常離線的presence證據，但Will內的state可能仍是連線當時的值，uptime_ms也可能為空；判斷故障前最後狀態時須另查最後一筆event、telemetry或本機log。

七、建立ESP32 MQTT程式

在檔案總管開 IOT_Introduction/examples/week12_mqtt_device，用 Arduino IDE 開[完整程式](#)，另存為個人 week12_mqtt_practice。不要把 Week 11 的 .ino 清空再貼新程式，保留上週可用版本。

Arduino IDE選擇Tools → Manage Libraries，搜尋PubSubClient，確認作者Nick O'Leary並安裝課程驗證版本2.8.0。保留ArduinoJson。

7.1 secrets.h

在 IDE 分頁右側選單選 **New Tab**，建立 secrets.h，貼入下列內容並替換字串。這次新增 broker 帳密；不能只沿用 Week 11 的 API 位址。

```
#pragma once
const char WIFI_SSID[] = "replace-with-wifi-name";
const char WIFI_PASSWORD[] = "replace-with-wifi-password";
const char MQTT_HOST[] = "192.168.1.23";
const int MQTT_PORT = 1883;
const char MQTT_USERNAME[] = "iotstudent";
const char MQTT_PASSWORD[] = "replace-with-broker-password";
```

MQTT_HOST只填筆電LAN IPv4，不加http://。本檔不得提交Git。

7.2 設定已開啟的完整程式

程式預設DRY_RUN=true及profile未填。先編譯，再抄入Week 3、5、7自己的實測值。

在 .ino 用 **Ctrl+F** 找 DEVICE_ID，填入與手機使用的同一個裝置代號。先保留 DRY_RUN=true 與 -1 腳位設定，按 **Ctrl+S** 後直接做第八節。[完整程式附錄](#)與已開啟的範例檔對應；上傳使用 IDE 中的完整 .ino。

八、分階段驗證單一裝置

階段1：編譯與連線，不驅動GPIO

先拔 USB、移除全部板端杜邦線及外部電源；只接板背 COM 的 USB。Arduino IDE 選 Week 11 同一板型與實際 Port，A、D、E 保持執行。

- 1. 保持DRY_RUN=true、GPIO為-1，替換DEVICE_ID及secrets.h。
- 2. Verify後Upload，開啟115200 Serial Monitor。
- 3. 正常依序看到wifi=connecting、mqtt=connect connected=true及本裝置topic。
- 4. mosquitto_sub訂閱course/+/presence，應立即看到retained online。

mqttClient.state()為負值時先記錄數字，再查broker、IP、port與帳密；不改硬體profile。

階段2：啟用實體profile

先沿用 Week 11 第九節的 START、STOP、RGB 接法，USB 拔除時只新增：

新增線路	程式設定
板 3V3 → a6；已確認 KY 中間電源腳 → b6	本批使用 3.3 V，不改接 5Vin
KY – → e3	與原有 a3 板 GND 同組，其他地線不移動
KY S → a15；c15 → 已確認 ADC GPIO	PIN_LIGHT；T01 為 GPIO4

- 1. 拍照保存此接法，再移除板端所有杜邦線，只接 USB。
- 2. 把 PIN_START、PIN_STOP、PIN_RGB_R、PIN_RGB_G、PIN_RGB_B、RGB_ON_LEVEL 填回 Week 11 值；PIN_LIGHT 依上表。LIGHT_VALID_MIN/MAX 分別採同輪兩種光線 基準的最小 / 最大值，涵蓋兩種條件；不能只填未遮光那一小段。改 DRY_RUN=false。
- 3. Verify、Upload 後拔 USB，依兩張表恢復接線，再接 USB、開 Monitor 選 115200。在 B 按 Ctrl+C 停掉原訂閱，再用下列命令訂閱本裝置，替換 IP、密碼及 <your-device-id>：

```
& "C:\Program Files\mosquitto\mosquitto_sub.exe" `
-h <筆電LAN-IP> -u iotstudent -P '<temporary-password>' `
-t "course/<your-device-id>/#" -v
```

- 4. 每兩秒應看到light_sample；按START / STOP應看到events。關閉broker後按STOP，RGB仍須在實測上限內變紅，且不等待broker恢復。

做完離線觀察，在 A 回原 MQTT 練習目錄，重新執行第四節的 -c .\course-mosquitto.conf -v 啟動命令；等 ESP32、B、E 重新連上。手機送 reset、等 done 與 idle，才做階段3。若 B 或 E 已退出，重跑它們原來的命令；新視窗要重填環境變數，不重建密碼檔。

`mqttClient.connect()` 本身仍可能阻塞到 `setSocketTimeout(1)` 設定的約1秒；因此本實驗只能 證明低功率 RGB範例的本機STOP不依賴broker，不能把它描述為硬即時緊急停止。量測broker 離線時的最壞STOP延遲並記錄。會移動或輸出高功率的專題必須另外設計不受network library 阻塞的停止路徑或獨立斷電裝置。

階段3：由手機送命令

1. 開Week 11 Backend頁面，填正確device ID與operator key。
2. 依序送reset、start、stop。
3. Bridge發布commands；ESP32發布accepted與terminal result到jacks。
4. 手機、bridge、broker與ESP32上的command_id必須一致。

本程式在RAM保留最近8個terminal command結果，仍在快取中的重複ID只重送原結果，不重複動作。第9筆結果會覆蓋最舊紀錄；被淘汰的ID即使未重開機也可能重做。重新開機會清除RAM，因此QoS 1與command_id仍不能單獨提供跨重啟exactly-once執行。會移動或輸出高功率的專題須另採可安全重做的命令、耐久完成紀錄、最大動作時間與 實體停止設計。

九、多裝置交叉驗證

與另一位學生配對，裝置ID記作A與B，不可相同。

1. 訂閱course/+/presence，確認A、B都online。
2. 訂閱course/+/telemetry，確認topic與payload ID各自一致。
3. 手機只對A送start；A變綠並ack，B不得改變。
4. 手機只對B送stop；B變紅並ack，A不得改變。
5. 拔A的USB，等待broker發布A的Last Will offline；B仍online。
6. A重新接USB後發布online，不能自動執行斷線前舊命令。

若兩片都執行，先查是否誤訂閱course/+/commands。裝置只能訂閱自己的commands。

十、Retain、重複命令與故障注入

1. A已online後才新開subscriber訂閱A presence；若立即收到online，retained生效。
2. 裝置離線時發布一個 不加 -r 的命令，重新上線後不得執行。
3. 將相同安全命令payload與command_id發布兩次；第二次不得重做狀態轉換。
4. 發布缺右大括號的malformed JSON；Serial顯示JSON reject，RGB不變。
5. Topic填A、payload device ID填B；A拒絕執行。
6. command填spin_forever；A回rejected。
7. 停止broker；實體STOP仍不依賴broker，程式不得被無限重連while卡住。量測最壞 反應時間，並確認未超過本組事先定義的低功率RGB測試上限。
8. 按住實體STOP不放並送reset；結果必須rejected，RGB保持紅色。放開STOP後才可 送出新的reset。
9. 只停止bridge；broker仍看得到資料，但Backend與手機不更新。

每次只注入一個錯誤，恢復baseline後再做下一項。

手動發布命令的方法

這段只對自己組的低功率 RGB 裝置操作。先完成手機命令測試，再於 **C 視窗** 執行。先在 E 按 Ctrl+C 暫停 bridge，避免手動測試 ID 的 ack 被誤當成資料庫命令；A 與 B 保持執行。

```
$device = "replace-with-your-device-id"
$testId = [guid]::NewGuid().ToString()
$commandBody = @{
    device_id = $device
    command_id = $testId
    command = "reset"
    parameters = @{}
} | ConvertTo-Json -Compress
$commandBody | & "C:\Program Files\mosquitto\mosquitto_pub.exe" `
    -h <筆電LAN-IP> -u iotstudent -P '<temporary-password>' `
    -t "course/$device/commands" -s
```

先替換裝置代號、IP、密碼；不加 -r。B 應收到同一 ID 的 accepted 與 done / rejected，並以 Serial、RGB 核對；這個自行產生的測試 ID 不會出現在手機 **Commands** 資料表。重複測試只重跑最後四行發布命令，不重建 \$testId。

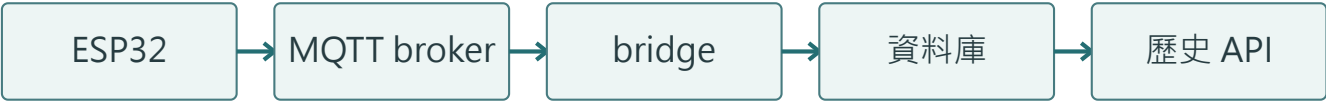
原有故障測試	C 中改哪裡	從哪裡看結果
格式錯誤	\$commandBody = '{"device_id":'，再發布	Serial 的 JSON reject；不是要求 broker 拒絕
目標不符	重建 body，裡面 ID 改另一個代號；topic 仍用自己 ID	Serial 顯示 mqtt=reject reason=identity_or_field，RGB 不變
未知命令	重建 body，command 改 spin_forever，使用新 \$testId	B 的 rejected 與原因
離線命令不重播	拔自己 ESP32 USB，發布一次新 ID 的 reset，再接 USB	重連後不出現該 ID 的執行紀錄

每項做完重建正常 body；測完在 E 重跑 bridge，從手機以新命令 reset，確認資料庫路徑恢復。不用任意 ID 覆寫資料庫，也不截圖含密碼的命令。新視窗沒有 \$device 等變數，須重做本段設定。

MQTT 收到了，不等於已入庫

資料保存示意；僅作圖上推演，不停止正在使用的服務或取消真實系統的安全檢查。

正常的連接 / 處理



只改標記的地方



箭頭表示資訊處理順序，不是供電或電流路徑。

想一想：保留 broker，mosquitto_sub 仍看得到新訊息，只停止 bridge。查歷史 API 時一定有這筆嗎？

原理提示：不一定；本課由 bridge 把 MQTT 訊息交給資料庫。看到 broker 訊息不是入庫證據。

只改圖中標記處，其餘條件保持相同。請指出哪一段仍工作，以及目前證據不能說明什麼。

十一、練習

練習1：Topic filter比較

比較course/+/telemetry、course/<自己的ID>/#與course/#的範圍。先寫預測，不使用可能接收其他系統資料的#單獨訂閱。

情境：自己的裝置代號為 team03-device01，只監看本課資料。以下是主題配對的預期，不是要求發送命令到其他組裝置。

發布主題	course/+/telemetry	course/team03-device01/#	course/#
course/team03-device01/telemetry	收到	收到	收到
course/team03-device01/events	不收到	收到	收到
course/team04-device01/telemetry	收到	不收到	收到

發布主題	course/+/telemetry	course/team03-device01/#	course/#
course/team04-device01/acks	不收到	不收到	收到

練習2：Presence原因

比較正常online的reason=mqtt_session與拔USB後offline的reason=last_will，說明為何使用同一presence topic與retained設定，仍可辨認來源。

練習3：多裝置隔離

把A、B、topic、payload ID與實體反應填入矩陣，至少包含正確與錯誤目標。以紀錄說明 broker依topic路由，裝置又用payload identity做第二次檢查。

十二、繳交內容與完成條件

繳交Broker、PubSubClient、ArduinoJson及board package版本；broker host與LAN test；完整topic tree；telemetry、presence、ack代表JSON；KY-018與按鈕 / RGB target test；一筆跨手機、bridge、broker、ESP32的命令ID；A / B隔離、Last Will、重連及舊命令不重播證據；以及全部故障注入紀錄。秘密必須遮蔽。

- ☐ Broker host test與有帳密LAN連線分別通過。
- ☐ 裝置使用唯一ID，只訂閱自己的commands topic。
- ☐ KY-018 telemetry含value、unit、valid、reason與uptime。
- ☐ Online與Last Will offline可由新subscriber立即讀取。
- ☐ Command不retain，最近8筆快取內的相同command ID不重複執行。
- ☐ A命令不改變B，錯誤payload identity也被拒絕。
- ☐ MQTT資料經bridge進入Backend、SQLite與手機WebSocket。
- ☐ Broker / bridge離線時，本機STOP不依賴服務恢復，且最壞反應時間已有實測值。
- ☐ 密碼只在環境變數或secrets.h，未提交Git。
- ☐ Host、compile、upload、network及physical target test分別標示。

前半段完成後保持低功率安全狀態，保存資料來源與時間範圍，接續下方DB章節；若中途離開，先正常停止服務並拔USB，回來依啟動順序重建。MQTT完整紀錄表、故障表與延伸挑戰見 [Week 12支援資料](#)。

DB 二、即時畫面、Database與Log不是同一件事

手機目前畫面：使用者現在看到什麼

Historical API：Backend允許查詢哪些已保存資料

SQLite Database：資料以哪些table、column與row保存

Structured log：程式在何時做了什麼判斷與處理

Database (資料庫) 有組織地保存資料。本例一個table (資料表) 存一類紀錄，一個row (列) 是一筆，一個column (欄位) 是一項內容；例如events表中的某筆事件有device_id欄位。關閉瀏覽器或重新啟動Backend後，已提交的SQLite資料仍存在。**log (日誌)** 按執行順序記錄程式行為，用來理解請求為何成功、被拒絕或失敗；log不一定等於正式歷史資料，也不能取代database schema。

以下範例使用SQLite：database存在單一檔案runtime/iot_course.db，適合本機課堂原型。這不代表所有正式IoT系統都應使用SQLite；多人寫入、備援、遠端部署與大量資料需要另外評估。

DB 三、開始狀態與資料責任

DB 必要環境

- 本週前半段的ESP32、KY-018、START、STOP及RGB可正常產生真實資料。
- IOT_Introduction/examples/course_backend的Python.venv已安裝requirements。
- Backend可啟動，手機可看到Events與Commands。
- 若使用MQTT路徑，broker與mqtt_bridge.py也要啟動；若只重現Week 11 HTTP路徑，可不啟動broker。

DB Database與log不得保存

- Wi-Fi密碼、broker密碼、operator key、API token。
- 姓名、學號、電話、電子郵件等本實驗不需要的個資。
- 完整HTTP authorization header或完整request body中的秘密。
- 無限制的高頻感測值；本週KY-018仍維持合理取樣間隔。

device_id使用課程代號，不含個資。錯誤log只記「權限不足」，不能把學生輸入的key印出來作為除錯方式。

DB 四、啟動可觀察的Backend

沿用 D 的 IOT_Introduction/examples/course_backend 資料夾。先拔 ESP32 USB，暫停產生資料；在 E 按 **Ctrl+C** 暫停轉接，在 D 按 **Ctrl+C** 停止後端。A 保持執行。不要在舊後端仍占用 8000 時啟動第二個後端。在 D 確認資料檔路徑：

```
Get-Location
Test-Path .\runtime\iot_course.db
```

第一次使用時database不存在是正常的，Backend啟動會建立。設定臨時operator key，並把畫面輸出同時保存成log：

```
New-Item -ItemType Directory -Force -Path .\runtime
$env:IOT_OPERATOR_KEY="replace-with-your-temporary-classroom-key"
.\.venv\Scripts\python.exe -m uvicorn app:app --host 0.0.0.0 --port 8000 2>&1 |
Tee-Object -FilePath .\runtime\week12_backend.log
```

看見 Uvicorn 啟動後，在原 E 視窗重跑 `.\.venv\Scripts\python.exe mqtt_bridge.py`。確認 `mqtt_connected`，才接回已確認安全接線的 ESP32。手機重新填同一個臨時 key 及 Device ID，按 **套用並重新整理**。如果 E 是新視窗，須先重填第五節四個 MQTT 變數與 API 位址。

structured log (結構化日誌) 把時間、動作與結果分成有名稱的欄位，方便查詢。本課每行使用一個JSON物件，例如：

```
{"timestamp":"2026-11-25T02:10:00+00:00","action":"event_created","event_id":21,"device_id":"team03-device01","event_type":"light_sample","valid":true,"reason":"within_profile"}
```

固定欄位使程式能篩選 `action`、`device_id` 或 `command_id`；自然語句「剛才好像有收到」無法提供同等查詢能力。Uvicorn 本身的啟動與 access log 可能不是 JSON，因此本週 log 檔是「包含 structured records 的程序 log」。JSONL 是每行一筆 JSON 的文字格式；混有其他啟動文字的整份紀錄不算純 JSONL。

DB 五、建立一組已知資料

分析前先索引本週 MQTT 階段已保存的小型資料集 (dataset)。若已具備以下事件、命令與時間證據，直接引用，不要求再量一次；只補缺少的類型。

1. 手機或 Backend 頁面 Device ID 填成自己的 ESP32 ID。
2. 讓 KY-018 在一般光線下至少送出五筆有效 `light_sample`。
3. 以遮光或照明方式安全改變環境，再送五筆；不拆帶電 signal 線製造變化。
4. 按 START 一次、STOP 一次、reset 一次。
5. 先送 STOP 使裝置進 ERROR，再送 start 取得 rejected；最後確認可安全 reset。
6. 記錄開始與結束時間、device ID，以及預期事件數。資料來源表放在 [Week 12 支援資料](#)。

若後半段實機暫時無法運作，可以使用前半段已保存的真實資料練習query，但必須標示資料 產生日期與來源；host test資料不得偽裝成physical target data。

DB 六、讀取SQLite Schema

schema (綱要) 是資料庫結構的規定：有哪些表、欄位、可存的資料型態及限制。例如events的id必須能唯一辨認一筆事件，不能兩筆共用同一id。

保持 A、B、D、E 執行，C 已發布完訊息，可用來查資料。在檔案總管從 IOT_Introduction/examples/course_backend 開新的查詢視窗，或將 C 切到此目錄。後面的 DB 命令都在這個查詢視窗執行，不貼進 D 的 log 畫面。

```
Test-Path .\inspect_db.py
.\.venv\Scripts\python.exe inspect_db.py schema
```

inspect_db.py以read-only mode開啟database，不修改row。輸出應包含events與 commands兩個table。第一行須為 True；直接呼叫 .venv 的 Python，不受 PowerShell 啟用腳本限制。後續 <your-device-id> 要換成本組代號，連尖括號一起移除；例如 --device team03-device01。

DB 6.1 Events table

時間欄位以ISO 8601格式寫日期與時間，例如2026-11-25T10:10:00+08:00；末尾是相對UTC的時區差。UTC是共同時間基準，這個例子對應UTC的02:10。null表示欄位沒有值，不是數字0。

Column	意義	為何存在
id	Backend產生的整數primary key	唯一辨認一筆事件
device_id	來源裝置	依裝置查詢
event_type	事件種類	區分light、button、presence等
value_json	JSON編碼後的值	可保存數字、文字或null
unit	值的單位	避免321失去意義
state	當時裝置狀態	解釋事件背景
valid	感測資料是否有效	無效值仍可保存供診斷
reason	有效 / 無效判斷原因	使判斷可追蹤
uptime_ms	裝置開機後毫秒	觀察裝置內相對順序
device_timestamp	裝置提供且含timezone offset的ISO 8601時間，可為null	只有裝置真的校時才使用
recorded_at	Backend收到時的UTC時間	歷史範圍查詢基準

primary key (主鍵) 是table中唯一辨認row的欄位。**id**不是感測值，也不是裝置ID。**nullable**表示欄位可為空；例如未做可靠校時時，**device_timestamp**保持null比填入假日期更正確。已校時的裝置也必須附Z或+08:00等timezone offset；沒有offset時Backend回422。建立event時，client送出的JSON欄位名是**timestamp**；Backend驗證後，在API response與SQLite中使用**device_timestamp**保存它。這個欄位名稱轉換不能和Backend產生的 **recorded_at**混為同一時間來源。

DB 6.2 Commands table

command_id是命令primary key；**device_id**是目標；**command**與**parameters_json**描述 要求；**status**從requested變成accepted，再到done / error / timeout / rejected；**requested_at**與**completed_at**保存生命週期。accepted尚未完成，因此**completed_at** 可以是null。

DB 6.3 Index

index (索引) 是database為常用查詢建立的搜尋結構。本範例對device/time、event type/time及command device/status建立index。index加速查詢，但會占空間並增加寫入 成本；不因「可能有用」就替每個column建立index。

DB 七、直接檢視Database Row

讀取最近十筆事件：

```
.\.venv\Scripts\python.exe inspect_db.py events --limit 10
```

只看自己的裝置與KY-018：

```
.\.venv\Scripts\python.exe inspect_db.py events --device <your-device-id> --type light_sample --limit 20
```

讀命令及只看被拒絕的命令：

```
.\.venv\Scripts\python.exe inspect_db.py commands --device <your-device-id> --limit 20
.\.venv\Scripts\python.exe inspect_db.py commands --device <your-device-id> --status rejected --limit 20
```

完成條件：從一筆row指出「誰、發生什麼、值與單位、是否有效、原因、裝置相對時間、Backend接收時間」。如果欄位為null，要說明是允許缺少、尚未完成，還是資料來源未提供。

DB 八、使用Historical API

直接讀database適合開發者檢查；手機與其他程式應透過Backend API取得經限制與驗證的資料，不共用database檔案。

DB 8.1 依裝置與事件種類查詢

```
$base = "http://127.0.0.1:8000"
$device = "<your-device-id>"
Invoke-RestMethod "$base/api/events?device_id=$device&event_type=light_sample&limit=20" |
    ConvertTo-Json -Depth 6
```

正常回傳JSON array；沒有符合資料時回空array[]，不是伺服器錯誤。since與until必須是含timezone的ISO 8601時間，而且since不可晚於until；格式錯誤、缺少timezone或範圍顛倒時Backend回422，不把錯誤filter當成空資料。

DB 8.2 依UTC時間範圍查詢

UTC是跨裝置交換時間常用的共同基準。recorded_at由Backend產生並含timezone offset。查詢最近十分鐘：

```
$since = (Get-Date).ToUniversalTime().AddMinutes(-10).ToString("o")
$encodedSince = [uri]::EscapeDataString($since)
Invoke-RestMethod "$base/api/events?device_id=$device&since=$encodedSince&limit=200" |
    ConvertTo-Json -Depth 6
```

uptime_ms不能直接轉成今天幾點；ESP32重新開機後它會從小值重新開始。若看到uptime突然變小但recorded_at繼續增加，可推測裝置曾restart，但必須再用boot log或presence佐證。

DB 8.3 查詢命令生命週期

```
Invoke-RestMethod "$base/api/commands?device_id=$device&limit=20" |
    ConvertTo-Json -Depth 6
```

選一個command_id，把requested、accepted與terminal result的log對齊。Database只保留目前status，因此完整中間過程要由structured log補充；這正是database與log分工。

DB 九、統計與解讀

DB 9.1 API統計

```
Invoke-RestMethod "$base/api/stats?device_id=$device" | ConvertTo-Json -Depth 6
```

輸出包含event count、invalid event count、event types、command count與command statuses。也可直接執行read-only summary：

```
.\.venv\Scripts\python.exe inspect_db.py summary
```

DB 9.2 不能混淆的三種「無效」

1. `valid=false`：格式正確、已保存，但裝置判定感測值不可信。
2. HTTP 422：request欄位不符合API，Backend拒絕，沒有新增event row。
3. 沒有資料：裝置未送、網路中斷、filter不符或時間範圍錯誤，不能自動推論為0。

DB 9.3 核心分析

從自己的已知dataset回答並附query：

- 兩種光線條件各有多少筆有效sample？
- invalid比例是多少？分母與分子各是什麼？
- 哪些command到達done，哪些rejected？
- 是否有requested或accepted後沒有terminal status？若有，它代表哪一段待查？
- recorded_at順序與uptime_ms順序是否一致？若不一致，需要哪項證據才能解釋？

只報百分比不算完成；必須寫出query條件、筆數、資料時間範圍與解讀限制。

DB 十、以Structured Log重建一次失敗

DB 10.1 製造可控制的422

使用不會被ESP32或bridge持續寫入的專用device ID，先記錄該ID目前event count，再送一筆uptime_ms=-1的無效request：

```
$validationDevice = "host-validation-test"
$before = Invoke-RestMethod "$base/api/stats?device_id=$validationDevice"
$before.event_count

$validationEvent = @{
    device_id = $validationDevice
    event_type = "light_sample"
    value = 123
    unit = "adc_raw"
    valid = $true
    reason = "intentional_validation_test"
    uptime_ms = -1
}
$badBody = $validationEvent | ConvertTo-Json

try {
    Invoke-RestMethod -Method Post -Uri "$base/api/events" `
        -ContentType application/json -Body $badBody
} catch {
    $_.Exception.Response.StatusCode.value__
}
```

預期status為422，Backend log出現request_validation_failed、path與欄位錯誤，但不印 Wi-Fi或 operator秘密。再次查event count，必須沒有因這筆request增加：

```
$afterReject = Invoke-RestMethod "$base/api/stats?device_id=$validationDevice"
$afterReject.event_count
```

接著只修正違反規則的欄位，再送出同一筆資料：

```
$validationEvent.uptime_ms = 1234
$correctBody = $validationEvent | ConvertTo-Json
$created = Invoke-RestMethod -Method Post -Uri "$base/api/events" `
    -ContentType application/json -Body $correctBody
$created

$afterCreate = Invoke-RestMethod "$base/api/stats?device_id=$validationDevice"
$afterCreate.event_count
```

afterReject.event_count應等於before.event_count；afterCreate.event_count應只增加1。若不同，先確認三次stats使用相同device ID，且沒有其他程序使用這個專用ID，不刪除row 去配合預期值。

DB 10.2 重建順序

依序記錄：

1. client在何時、對哪個path送出request；
2. HTTP status；
3. structured log的action與欄位錯誤；
4. database count前後差異；
5. 根因：uptime_ms違反非負整數規則；
6. 修正後同一資料改成合理非負值，再送一次取得201；
7. 新event ID與recorded time。

這是一個可重現、可定位、可修正、可驗證復原的故障描述。只寫「JSON錯了」不足。

DB 十一、Schema Migration的基本判斷

migration (資料庫遷移) 是在保留既有資料的前提下調整schema。本範例init_db()先 CREATE TABLE IF NOT EXISTS，再以ensure_column()檢查舊database是否缺少新column；重複啟動不應重複新增相同column，這稱為idempotent (重複執行仍得到相同結構結果)。

本週不直接修改正式database schema。完成下列read-only觀察：

1. 執行`.\.venv\Scripts\python.exe inspect_db.py schema`保存第一次輸出。
2. 正常停止並重新啟動Backend。
3. 再執行schema，確認column沒有重複、既有event row仍可查詢。
4. 記錄「重新啟動驗證」；不能把它誤稱成完整migration test，因為沒有建立舊版fixture。

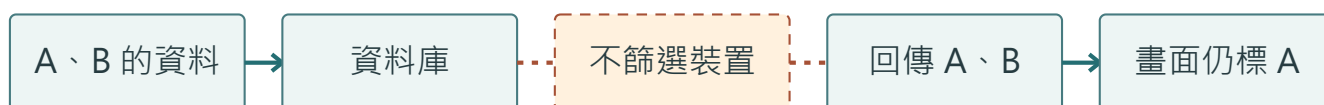
少了裝置篩選，查到的是誰的資料？

查詢資訊流示意；僅作圖上推演，不停止正在使用的服務或取消真實系統的安全檢查。

正常的連接 / 處理



只改標記的地方



箭頭表示資訊處理順序，不是供電或電流路徑。

想一想： 資料不變，只刪除 API 查詢的 `device_id` 條件。總數增加，可以說 A 的事件增加了嗎？

原理提示： 不能。查詢範圍變了，可能混入 B；數字增加不代表 A 產生更多事件。

只改圖中標記處，其餘條件保持相同。請指出哪一段仍工作，以及目前證據不能說明什麼。

DB 十二、練習

DB 練習1：設計一個可驗證的filter

先固定一段不再追加資料的來源或時段，兩種查詢使用相同 `device`、`event type` 與 `limit`，並比較每筆 `id`，不只比較畫面行數。若來源仍持續增加，先記錄查詢先後時間，不能把新增資料當成查詢錯誤。

選一個 `device` 與 `event type`，先預測筆數，再用 API 與 `inspect_db.py` 各查一次。若結果不同，比較 `limit`、排序、時間範圍及資料解碼，不手動改 `row`。

DB 練習2：找出未完成命令

查 `status` 為 `requested` 或 `accepted` 的命令，選一筆從 Backend log 往前後追蹤。若沒有未完成命令，停止 ESP32 後建立一筆測試命令，觀察 `requested`，再重新上線完成或明確標為測試未執行。

DB 練習3：資料最小化

檢查自己的 `event`、`command` 與 `log` 欄位，列出「完成故障重建真正需要」與「不需要且不應保存」的資料各三項，說明理由。

DB 十三、繳交內容與完成條件

繳交：database檔路徑與備份策略說明（不提交runtime database）、events / commands schema、已知資料集來源、三種historical query、stats結果與解讀、一次成功command trace、一次422 完整重建、重新啟動後schema與資料仍存在的證據，以及秘密 / 個資檢查。

- ☐ Events與commands的key、column、nullable欄位與時間來源可正確解釋。
- ☐ 真實ESP32資料可由API依device、event type及時間範圍查詢。
- ☐ API與read-only database查詢在相同條件下可對照。
- ☐ 統計保留valid、status與時間範圍，不把缺資料當0。
- ☐ 一筆command可由ID連結request、log、database status及實體結果。
- ☐ 422 request未入庫，修正後201入庫，count前後證據完整。
- ☐ Database重新啟動後schema未重複、既有row仍存在。
- ☐ Database與log不含password、key、token或不必要個資。
- ☐ Host query與physical data來源分別標示，未做的測試不宣稱通過。

結束先放開 STOP 並送 reset，確認安全狀態、拔 ESP32 USB；在 E、B、D、A 依序按 Ctrl+C 停止 bridge、訂閱、Backend、broker。下次依第四、五節重開，不重新安裝或覆寫密碼檔。不在Backend執行時搬移 database。runtime 資料不提交Git；只提交遮蔽秘密的schema、query、統計及重建紀錄。詳細表格與延伸題見 [Week 12支援資料](#)。

準備、紀錄表與延伸參考

一、課前軟體與版本紀錄

項目	取得方式	實際版本 / 路徑	開始條件
Eclipse Mosquitto	mosquitto.exe -h首行		broker、pub、sub三個程式存在
Python	python --version		Week 11環境可用
paho-mqtt	python -m pip show paho-mqtt		course backend環境已安裝
PubSubClient	Arduino Library Manager		課程驗證版本2.8.0
ArduinoJson	Arduino Library Manager		Week 11相同版本
ESP32 board package	Boards Manager		Week 11相同版本
Backend tests	python -m pytest -q		host test結果已記錄

安裝路徑不同不代表錯誤；教材命令中的路徑必須換成自己的實際路徑。不要在同一次 故障排除中同時升級 broker、Arduino library與board package。

二、Topic與Payload唯一規格表

用途	Topic	Publisher	Subscriber	必要JSON欄位	Retained
Presence	course/<id>/presence	ESP32 / broker Last Will	bridge、monitor	device_id、 event_type、 value、state、 valid、reason； Will的uptime可為空	是
Telemetry	course/<id>/telemetry	ESP32	bridge、monitor	device_id、 event_type、 value、unit、 valid、reason、 uptime_ms	否
Event	course/<id>/events	ESP32	bridge、monitor	device_id、 event_type、 value、unit、 state、valid、 reason、 uptime_ms	否
Command	course/<id>/commands	bridge	目標ESP32	device_id、 command_id、 command、 parameters	否
Ack / result	course/<id>/acks	ESP32	bridge、monitor	device_id、 Backend產生的 UUID command_id、 result、message	否

自己的Topic展開表

```

DEVICE_ID：_____
Telemetry：course/_____/telemetry
Events：   course/_____/events
Presence： course/_____/presence
Commands： course/_____/commands
Acks：     course/_____/acks

```

禁止的ID例子及原因：

例子	問題
王小明	含個資
team 3	含空白，容易在命令與topic中不一致

例子	問題
team3/device1	/會改變topic階層
team+	+是subscription wildcard
#3	#是多層wildcard

三、訊息觀察表

Presence

裝置	訂閱開始時間	連線方式	value	reason	retained立即收到	結論
A		正常上線	online			
A		拔USB	offline			
B		正常上線	online			

Telemetry取樣

device_id	topic	raw value	unit	valid	reason	uptime_ms	Backend event id

若topic中的ID與payload不同、event type含不支援字元，或ack的command ID不是Backend 產生的UUID，bridge應拒絕。不要手動改database讓錯誤資料看似成功。

Command與Ack

command_id	device	command	publish topic	accepted	terminal result	physical result	duplicate result
		start					
		stop					
		unknown					

四、多裝置隔離矩陣

測試	Topic target	Payload ID	A應反應	B應反應	A實際	B實際	結論
正確控制A	A	A	執行並ack	不變			
正確控制B	B	B	不變	執行並ack			

測試	Topic target	Payload ID	A應反應	B應反應	A實際	B實際	結論
Topic / payload不一致	A	B	拒絕	不變			
不存在裝置	C	C	不變	不變			
相同 command ID重送A	A	A	不重做動作	不變			

「A、B都收到subscriber畫面的資料」不代表兩片都執行命令。實體反應、Serial log與ack 三項都要觀察。

五、Broker與Bridge程序地圖

建議在各PowerShell標題前人工加上A～E標籤，避免把命令輸入錯誤視窗：

視窗	程序	保持執行時的正常輸出	停止方式
A	Mosquitto broker	connection、subscribe、publish log	Ctrl+C
B	mosquitto_sub monitor	topic與payload	Ctrl+C
C	手動mosquitto_pub	成功時通常立即結束	不需常駐
D	FastAPI Backend	request與structured log	Ctrl+C
E	mqtt_bridge.py	connected、forwarded、command published	Ctrl+C

若E停止但A仍收到資料，表示ESP32到broker正常，故障在bridge以後；若A完全沒有ESP32 connection，先查Wi-Fi / broker，不查database。

六、故障排查表

症狀	層次	第一個檢查	正常基準	安全復原
Broker啟動後立即關閉	broker config	讀第一個error行	顯示listener 1883	修正路徑 / port，不改ESP32
Address already in use	broker process	查是否已有broker視窗	只有一個listener	停止重複程序，不殺未知服務
not authorised	authentication	核對username與密碼檔	client connected	重新輸入密碼，不截圖
ESP32 MQTT state負值	MQTT connection	記錄state數字與broker log	connected=true	核對host、port、帳密

症狀	層次	第一個檢查	正常基準	安全復原
ESP32有Wi-Fi但broker無連線	network/broker	比對MQTT_HOST與筆電LAN IP	broker看到client ID	查private firewall
Presence online但無telemetry	device/profile	拔USB後核對DRY_RUN與KY-018	每2秒一筆	不先改broker
Bridge rejected	validation	讀error與原topic	topic/payload ID一致	修正publisher，不繞過驗證
Broker有資料，Backend沒有	bridge/API	看E是否connected	mqtt_forwarded	啟動Backend再bridge
命令停requested	relay	看bridge command published	topic為目標ID	查E與broker
有command topic但裝置不動	subscriber/validation	看ESP32 reject reason	訂閱自己的topic	核對payload欄位
兩片都執行	topic isolation	查訂閱topic	僅自己的commands	移除wildcard command訂閱
重連後執行舊命令	retained command	檢查command是否retain	command retain=false	清除retained舊值並修publisher
拔USB沒有offline	Last Will	等待keepalive後看broker	retained offline	核對connect的will topic
Broker離線後STOP無反應或過慢	local safety	拔USB	STOP不依賴MQTT；延遲在實測上限內	移除無限重連，設定socket timeout；高功率另設獨立停止

清除意外retained command時，必須指定**確切topic**發布零長度retained payload；先讓所有裝置斷電並由教師確認topic，避免誤清除整個課程tree。學生不得對course/#做批次刪除。

七、故障注入紀錄

故障	唯一變因	預測	Broker觀察	ESP32觀察	Backend觀察	復原證據
Malformed JSON						
Wrong payload ID						
Unknown command						
Broker停止						
Bridge停止						

八、秘密與提交前檢查

- ☐ secrets.h未被Git追蹤。
- ☐ Mosquitto password file與設定檔不在repository。
- ☐ PowerShell截圖沒有-P後方密碼。
- ☐ MQTT password與operator key沒有出現在Serial Monitor。
- ☐ Broker只開private LAN，沒有router port forwarding。
- ☐ DEVICE_ID不含個資。
- ☐ Git diff中沒有真實SSID、host公網位址或密碼。

九、Lab Notebook模板

日期 / 組別 / commit :

Host tests :

- Mosquitto local pub/sub :
- Authenticated LAN pub/sub :
- Backend pytest :
- MQTT bridge connected :

Target :

- Board package / libraries :
- DEVICE_ID :
- Week 3、5、7 profile來源 :
- Compile :
- Upload :
- Physical telemetry / input / output :

Multi-device :

- A :
- B :
- 正確路由 :
- 錯誤路由拒絕 :

Presence :

- retained online :
- Last Will offline :
- reconnect :

Command trace :

- command_id :
- requested / accepted / terminal :
- duplicate behavior :

故障與復原 :

十、延伸實作

延伸A：DHT11與不同event type

沿用Week 4 DHT11 profile，發布temperature_sample與humidity_sample。保留最小取樣 間隔、無效值與reason；不可為了MQTT畫面更新而過度取樣。

延伸B：群組監看而非群組控制

建立course/+/telemetry監看頁，允許同時看所有裝置；commands仍必須精確指定裝置，不可使用wildcard控制。

延伸C：比較HTTP與MQTT

用同一筆KY-018事件比較Week 11 HTTP POST與Week 12 MQTT publish的sender、receiver、routing、response / ack、offline行為。結論須由實驗證據支持，不只列名詞。

延伸D：Presence時間過期

在前台除了online/offline，再顯示Backend最後收到presence的時間。設計「多久未更新視為 stale」的規則，並區分stale與broker明確發布offline。

十一、參考資料

- [Eclipse Mosquitto官方下載](#)
- [Mosquitto發布工具：-s從標準輸入讀取訊息](#)
- [PubSubClient官方repository與限制](#)
- [課程MQTT bridge](#)
- [課程Backend執行說明](#)
- [Week 11主教材](#)

DB 一、資料與時間來源表

欄位	產生者	時鐘 / 來源	可否為null	正確用途	不可做的推論
event id	Backend / SQLite	自增整數	否	唯一辨認row	不代表裝置時間
device_id	ESP32 / command client	profile常數	否	區分裝置	不含個資
uptime_ms	ESP32	開機後單調時間	是	裝置內相對順序	不等於日期時間
device_timestamp	ESP32	只有可靠校時才填，且含 timezone offset	是	裝置觀察時間	未校時不可假填；無offset會被拒絕
recorded_at	Backend	Server UTC	否	歷史query基準	不等於感測瞬間
requested_at	Backend	Server UTC	否	命令建立時間	不代表裝置收到

欄位	產生者	時鐘 / 來源	可否為null	正確用途	不可做的推論
completed_at	Backend	Server UTC	是	terminal result時間	accepted時可為null

DB 二、已知資料集紀錄表

項目	開始時間UTC	結束時間UTC	Device ID	預期筆數	實際筆數	資料來源 / 證據
一般光線 sample				5		
改變光線 sample				5		
START event				1		
STOP event				1		
reset command				1		
rejected start				1		

實際筆數不同時先檢查取樣週期、開始 / 結束邊界、重送與filter，不直接刪除「多出來」的row。

DB 三、Schema閱讀表

DB Events

Column	SQLite type	notnull	primary key	來源	一筆實際值	解釋
id						
device_id						
event_type						
value_json						
unit						
state						
valid						
reason						
uptime_ms						

Column	SQLite type	notnull	primary key	來源	一筆實際值	解釋
device_timest amp						
recorded_at						

DB Commands

Column	SQLite type	notnull	primary key	來源	一筆實際值	解釋
command_id						
device_id						
command						
parameters_js on						
status						
message						
requested_at						
completed_at						

DB 四、Historical Query紀錄

Query目的	API path及 parameters	預測筆數	回傳筆數	第一筆 / 最後一筆 時間	結論
本裝置最近事件					
本裝置light sample					
最近十分鐘					
rejected commands					
無符合資料		0			

API預設有limit與排序。比較API和direct database時，必須讓device、type、status、time range與limit一致。

DB 五、統計工作表

分析device：
 時間範圍：
 query / filter：

全部event數：
 valid=true：
 valid=false：
 valid為null：
 invalid比例（分子 / 分母）：

command總數：
 requested：
 accepted：
 done：
 rejected：
 error：
 timeout：

資料限制：
 不能由這組資料回答的問題：

DB 三種「沒有成功資料」比較

情況	HTTP status	是否新增event row	應如何呈現
格式正確且valid=true	201	是	有效sample
格式正確但valid=false	201	是	保存但標示無效及reason
欄位驗證失敗	422	否	request error，不是假sample
Query無符合資料	200	否	空array，不是value=0
裝置未送達	無event request	否	missing / offline，原因待查

DB 六、Command Reconstruction表

檢查點	時間	command_id	status / action	device	message / physical evidence
手機建立			requested		
Backend log			command_created		
Device收到			accepted		
Device決定			done / rejected / error		

檢查點	時間	command_id	status / action	device	message / physical evidence
Backend更新			command_result		
Database row			terminal status		
Mobile WebSocket			final display		

若database只有terminal status，accepted的中間狀態應從structured log或ESP32 log取得，不能自行補一個不存在的database row。

DB 七、422故障重建表

本表使用主教材指定的host-validation-test，避免ESP32持續telemetry改變計數。

步驟	觀察位置	預期	實際	證據定位
Count before	/api/stats	記錄N		
Bad request	client	uptime=-1		
HTTP response	client	422		
Validation log	Backend	path與欄位規則		
Count after rejection	/api/stats	仍為N		
Corrected request	client	non-negative uptime		
Correct response	client	201與event id		
Final count	/api/stats	N+1		

DB 八、故障排查表

症狀	第一個檢查	最可能層次	正常基準	下一步
Database不存在	看Backend是否至少啟動一次	initialization	runtime中有db	啟動Backend，不手建空檔
database is locked	看是否有多個程序使用db	process/database	一個Backend writer	正常停止重複程序
Inspect script回error	讀完整database path	file/path	read-only connect成功	核對IOT_DB_PATH
API有資料，direct query無	比較兩者DB path	environment	指向同一檔案	清除錯誤env後重啟

症狀	第一個檢查	最可能層次	正常基準	下一步
Filter回空array	移除一個filter重試	query	已知資料可查	核對ID、type、UTC範圍
Count比預期多	查time range與取樣率	dataset	已知開始 / 結束	不直接刪row
Count比預期少	看device / bridge log	transport	每次POST / forward 成功	定位缺失層次
valid都是null	查舊資料或sender schema	data quality	新資料明確true/false	不把null算false
Command停 requested	查device routing	command flow	有accepted / terminal	回Week 11及本週命令流程追蹤
Log找不到command ID	確認本次log檔與程序	observability	同一執行期間	不拼湊不同run
Log出現秘密	立即停止分享	data protection	只記denied原因	清除 / 重設秘密並修log

DB 九、Database備份與復原原則

本週不提交runtime/iot_course.db，也不直接編輯正式row。需要snapshot時：

1. 先以Ctrl+C正常停止Backend與bridge。
2. 確認程序停止後，將database複製成有日期的snapshot，不搬移原檔。
3. Snapshot放在不提交Git的位置，記錄來源commit與時間。
4. 復原演練使用snapshot副本，不覆寫唯一原檔。
5. 若資料含其他同學裝置ID，分享前先取得授權並最小化資料。

DB 十、Lab Notebook模板

日期 / 組別 / commit：

Backend資料庫路徑：

Log路徑：

已知資料集：

- 來源硬體：
- device ID：
- UTC範圍：
- 預期 / 實際筆數：

Schema：

- events key與重要columns：
- commands key與重要columns：
- nullable理由：

Queries：

- by device：
- by type：

```
- by time :  
- command status :  
  
Analysis :  
- 計數與範圍 :  
- invalid定義 :  
- 限制 :  
  
Reconstruction :  
- identity :  
- sequence :  
- root cause :  
- correction :  
- recovery evidence :  
  
秘密 / 個資檢查 :
```

DB 十一、延伸實作

DB 延伸A：歷史折線資料

只使用valid=true的light_sample，依recorded_at排序輸出time與valuearray。保留 unit與filter條件；不把無效值靜默轉成0。

DB 延伸B：Command completion rate

定義terminal status集合及分母，計算done比例。另列requested / accepted未完成筆數，說明 短時間內觀察與長時間stuck command的差異。

DB 延伸C：Restart推測

找出同device中uptime_ms突然變小的位置，再以presence、boot event或Serial log佐證。沒有第二項證據時只寫「可能restart」，不能寫成確定事實。

DB 延伸D：Schema migration fixture

在正式database之外建立最小舊版fixture，執行init_db()後比較schema與原row。這才可記為 migration host test；不得以重新啟動最新版database取代。

DB 十二、Repository參考

- [Backend API與schema](#)
- [Read-only database inspector](#)
- [Backend執行與驗證](#)
- [Backend host tests](#)
- [Week 12 MQTT主教材](#)

附錄：完整 MQTT 程式

這裡供完成操作後閱讀；第一次請依第七節開啟範例檔，不需手動複製這段。

```
#include <Arduino.h>
#include <ArduinoJson.h>
#include <PubSubClient.h>
#include <WiFi.h>
#include "secrets.h"

const char DEVICE_ID[] = "replace-with-team-device-id";
const bool DRY_RUN = true;
const int PIN_START = -1;
const int PIN_STOP = -1;
const int PIN_LIGHT = -1;
const int PIN_RGB_R = -1;
const int PIN_RGB_G = -1;
const int PIN_RGB_B = -1;
const int RGB_ON_LEVEL = -1;
const int LIGHT_VALID_MIN = -1;
const int LIGHT_VALID_MAX = -1;

enum class DeviceState { IDLE, ACTIVE, ERROR_STATE };
DeviceState state = DeviceState::IDLE;
WiFiClient wifiClient;
PubSubClient mqttClient(wifiClient);

String topicTelemetry, topicEvents, topicPresence, topicCommands, topicAcks;
bool startLastRaw = false, stopLastRaw = false;
bool startStablePressed = false, stopStablePressed = false;
unsigned long startChangedAt = 0, stopChangedAt = 0;
unsigned long lastWifiAttemptAt = 0, lastMqttAttemptAt = 0, lastTelemetryAt = 0;
const unsigned long DEBOUNCE_MS = 35;
const unsigned long WIFI_RETRY_MS = 10000;
const unsigned long MQTT_RETRY_MS = 5000;
const unsigned long TELEMETRY_MS = 2000;

struct ProcessedCommand {
    String id;
    String result;
    String message;
};
const int PROCESSED_COMMAND_CAPACITY = 8;
ProcessedCommand processedCommands[PROCESSED_COMMAND_CAPACITY];
int nextProcessedCommand = 0;

const char *stateName() {
    switch (state) {
        case DeviceState::IDLE: return "idle";
        case DeviceState::ACTIVE: return "active";
        case DeviceState::ERROR_STATE: return "error";
    }
    return "unknown";
}
```

```

bool identifierReady(const char *value) {
    size_t length = strlen(value);
    if (length == 0 || length > 80 || String(value).startsWith("replace-")) return false;
    for (size_t index = 0; index < length; index++) {
        char character = value[index];
        bool allowed = isAlphaNumeric(character) || character == '.' ||
                       character == '_' || character == '-';
        if (!allowed) return false;
    }
    return isAlphaNumeric(value[0]);
}

bool allPinsUnique(const int *pins, size_t count) {
    for (size_t left = 0; left < count; left++)
        for (size_t right = left + 1; right < count; right++)
            if (pins[left] == pins[right]) return false;
    return true;
}

bool profileReady() {
    const int pins[] = {
        PIN_START, PIN_STOP, PIN_LIGHT, PIN_RGB_R, PIN_RGB_G, PIN_RGB_B
    };
    bool pinsReady = PIN_START >= 0 && PIN_STOP >= 0 && PIN_LIGHT >= 0 &&
                     PIN_RGB_R >= 0 && PIN_RGB_G >= 0 && PIN_RGB_B >= 0;
    bool range = LIGHT_VALID_MIN >= 0 && LIGHT_VALID_MAX <= 4095 &&
                 LIGHT_VALID_MAX > LIGHT_VALID_MIN;
    bool level = RGB_ON_LEVEL == HIGH || RGB_ON_LEVEL == LOW;
    return pinsReady && allPinsUnique(pins, 6) && range && level;
}

int rgbOffLevel() { return RGB_ON_LEVEL == HIGH ? LOW : HIGH; }

void setRgb(bool red, bool green, bool blue) {
    if (DRY_RUN || !profileReady()) return;
    digitalWrite(PIN_RGB_R, red ? RGB_ON_LEVEL : rgbOffLevel());
    digitalWrite(PIN_RGB_G, green ? RGB_ON_LEVEL : rgbOffLevel());
    digitalWrite(PIN_RGB_B, blue ? RGB_ON_LEVEL : rgbOffLevel());
}

void enterState(DeviceState next, const char *reason) {
    state = next;
    if (state == DeviceState::IDLE) setRgb(false, false, true);
    if (state == DeviceState::ACTIVE) setRgb(false, true, false);
    if (state == DeviceState::ERROR_STATE) setRgb(true, false, false);
    Serial.printf("state=%s reason=%s\n", stateName(), reason);
}

void buildTopics() {
    String root = "course/" + String(DEVICE_ID) + "/";
    topicTelemetry = root + "telemetry";
    topicEvents = root + "events";
    topicPresence = root + "presence";
    topicCommands = root + "commands";
    topicAcks = root + "acks";
}

bool publishJson(const String &topic, JsonDocument &document, bool retained = false) {
    if (!mqttClient.connected()) return false;

```

```

String payload;
serializeJson(document, payload);
bool sent = mqttClient.publish(topic.c_str(), payload.c_str(), retained);
Serial.printf("mqtt=publish topic=%s sent=%s payload=%s\n",
              topic.c_str(), sent ? "true" : "false", payload.c_str());
return sent;
}

void publishPresence(const char *value) {
    JsonDocument doc;
    doc["device_id"] = DEVICE_ID; doc["event_type"] = "presence";
    doc["value"] = value; doc["unit"] = "status"; doc["state"] = stateName();
    doc["valid"] = true; doc["reason"] = "mqtt_session"; doc["uptime_ms"] = millis();
    publishJson(topicPresence, doc, true);
}

void publishEvent(const char *type, int value, const char *unit,
                 bool valid, const char *reason) {
    JsonDocument doc;
    doc["device_id"] = DEVICE_ID; doc["event_type"] = type;
    doc["value"] = value; doc["unit"] = unit; doc["state"] = stateName();
    doc["valid"] = valid; doc["reason"] = reason; doc["uptime_ms"] = millis();
    publishJson(topicEvents, doc);
}

void publishAck(const String &id, const char *result, const char *message) {
    JsonDocument doc;
    doc["device_id"] = DEVICE_ID; doc["command_id"] = id;
    doc["result"] = result; doc["message"] = message;
    publishJson(topicAcks, doc);
}

int findProcessedCommand(const String &id) {
    for (int index = 0; index < PROCESSED_COMMAND_CAPACITY; index++) {
        if (processedCommands[index].id == id) return index;
    }
    return -1;
}

void finishCommand(const String &id, const char *result, const char *message) {
    processedCommands[nextProcessedCommand] = {id, result, message};
    nextProcessedCommand = (nextProcessedCommand + 1) % PROCESSED_COMMAND_CAPACITY;
    publishAck(id, result, message);
}

void executeCommand(const String &id, const String &command) {
    int previousIndex = findProcessedCommand(id);
    if (previousIndex >= 0) {
        publishAck(id, processedCommands[previousIndex].result.c_str(),
                  processedCommands[previousIndex].message.c_str());
        return;
    }
    publishAck(id, "accepted", "received by device");
    if (command == "stop") {
        enterState(DeviceState::ERROR_STATE, "remote_stop");
        finishCommand(id, "done", "safe output applied");
    } else if (command == "start" && state != DeviceState::ERROR_STATE) {
        enterState(DeviceState::ACTIVE, "remote_start");
        finishCommand(id, "done", "active output applied");
    }
}

```

```

    } else if (command == "start") {
        finishCommand(id, "rejected", "reset required after error");
    } else if (command == "reset") {
        if (!DRY_RUN && stopStablePressed) {
            finishCommand(id, "rejected", "release physical stop before reset");
        } else {
            enterState(DeviceState::IDLE, "remote_reset");
            finishCommand(id, "done", "idle output applied");
        }
    } else {
        finishCommand(id, "rejected", "unknown command");
    }
}

void onMqttMessage(char *topic, byte *payload, unsigned int length) {
    if (String(topic) != topicCommands) return;
    if (length == 0 || length >= 768) {
        Serial.printf("mqtt=reject reason=payload_length length=%u\n", length); return;
    }
    char buffer[768];
    memcpy(buffer, payload, length); buffer[length] = '\0';
    JsonDocument doc;
    DeserializationError error = deserializeJson(doc, buffer);
    if (error) {
        Serial.printf("mqtt=reject reason=json detail=%s\n", error.c_str()); return;
    }
    String target = doc["device_id"] | "";
    String id = doc["command_id"] | "";
    String command = doc["command"] | "";
    if (target != DEVICE_ID || id.length() == 0 || command.length() == 0) {
        Serial.println("mqtt=reject reason=identity_or_field"); return;
    }
    executeCommand(id, command);
}

void requestWifi() {
    if (WiFi.status() == WL_CONNECTED) return;
    unsigned long now = millis();
    if (lastWifiAttemptAt && now - lastWifiAttemptAt < WIFI_RETRY_MS) return;
    lastWifiAttemptAt = now; WiFi.disconnect(); WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
    Serial.println("wifi=connecting");
}

void requestMqtt() {
    if (WiFi.status() != WL_CONNECTED || mqttClient.connected()) return;
    unsigned long now = millis();
    if (lastMqttAttemptAt && now - lastMqttAttemptAt < MQTT_RETRY_MS) return;
    lastMqttAttemptAt = now;

    JsonDocument willDoc;
    willDoc["device_id"] = DEVICE_ID; willDoc["event_type"] = "presence";
    willDoc["value"] = "offline"; willDoc["unit"] = "status";
    willDoc["state"] = stateName(); willDoc["valid"] = true;
    willDoc["reason"] = "last_will";
    String willPayload; serializeJson(willDoc, willPayload);

    bool connected = mqttClient.connect(
        DEVICE_ID, MQTT_USERNAME, MQTT_PASSWORD,
        topicPresence.c_str(), 1, true, willPayload.c_str());
}

```

```

Serial.printf("mqtt=connect connected=%s state=%d\n",
              connected ? "true" : "false", mqttClient.state());
if (connected) {
    mqttClient.subscribe(topicCommands.c_str(), 1);
    publishPresence("online");
}
}

bool pressedEvent(int pin, bool &lastRaw, bool &stablePressed,
                  unsigned long &changedAt) {
    bool pressed = digitalRead(pin) == LOW;
    unsigned long now = millis();
    if (pressed != lastRaw) {
        lastRaw = pressed;
        changedAt = now;
    }
    if (now - changedAt >= DEBOUNCE_MS && pressed != stablePressed) {
        stablePressed = pressed;
        return stablePressed;
    }
    return false;
}

void readPhysicalInputs() {
    if (DRY_RUN || !profileReady()) return;
    bool stopPressedEvent =
        pressedEvent(PIN_STOP, stopLastRaw, stopStablePressed, stopChangedAt);
    bool startPressedEvent =
        pressedEvent(PIN_START, startLastRaw, startStablePressed, startChangedAt);
    if (stopStablePressed) {
        if (stopPressedEvent || state != DeviceState::ERROR_STATE) {
            enterState(DeviceState::ERROR_STATE, "physical_stop");
            publishEvent("stop_pressed", 1, "pressed", true, "physical_input");
        }
        return;
    }
    if (startPressedEvent) {
        if (state == DeviceState::ERROR_STATE)
            publishEvent("start_rejected", 1, "pressed", false, "reset_required");
        else {
            enterState(DeviceState::ACTIVE, "physical_start");
            publishEvent("start_pressed", 1, "pressed", true, "physical_input");
        }
    }
}

void publishTelemetryIfDue() {
    if (DRY_RUN || !profileReady() || !mqttClient.connected()) return;
    unsigned long now = millis();
    if (now - lastTelemetryAt < TELEMETRY_MS) return;
    lastTelemetryAt = now;
    int raw = analogRead(PIN_LIGHT);
    bool valid = raw >= LIGHT_VALID_MIN && raw <= LIGHT_VALID_MAX;
    JsonDocument doc;
    doc["device_id"] = DEVICE_ID; doc["event_type"] = "light_sample";
    doc["value"] = raw; doc["unit"] = "adc_raw"; doc["state"] = stateName();
    doc["valid"] = valid;
    doc["reason"] = valid ? "within_profile" : "out_of_profile";
    doc["uptime_ms"] = now;
}

```

```

    publishJson(topicTelemetry, doc);
}

void setup() {
    Serial.begin(115200); delay(500); buildTopics();
    if (!identifierReady(DEVICE_ID)) {
        Serial.println("fatal=device_id_missing_or_invalid"); return;
    }
    if (!DRY_RUN && !profileReady()) {
        Serial.println("fatal=hardware_profile_incomplete"); return;
    }
    if (!DRY_RUN) {
        pinMode(PIN_START, INPUT_PULLUP); pinMode(PIN_STOP, INPUT_PULLUP);
        pinMode(PIN_RGB_R, OUTPUT); pinMode(PIN_RGB_G, OUTPUT); pinMode(PIN_RGB_B, OUTPUT);
        enterState(DeviceState::IDLE, "boot");
    }
    WiFi.mode(WIFI_STA);
    mqttClient.setServer(MQTT_HOST, MQTT_PORT);
    mqttClient.setCallback(onMqttMessage);
    mqttClient.setBufferSize(768);
    mqttClient.setKeepAlive(20);
    mqttClient.setSocketTimeout(1);
    requestWifi();
    Serial.printf("week=12 device=%s topic=%s mode=%s\n",
        DEVICE_ID, topicCommands.c_str(), DRY_RUN ? "dry_run" : "hardware");
}

void loop() {
    readPhysicalInputs(); // broker離線時也先處理本機STOP
    requestWifi();
    requestMqtt();
    if (mqttClient.connected()) mqttClient.loop();
    publishTelemetryIfDue();
    delay(5);
}

```